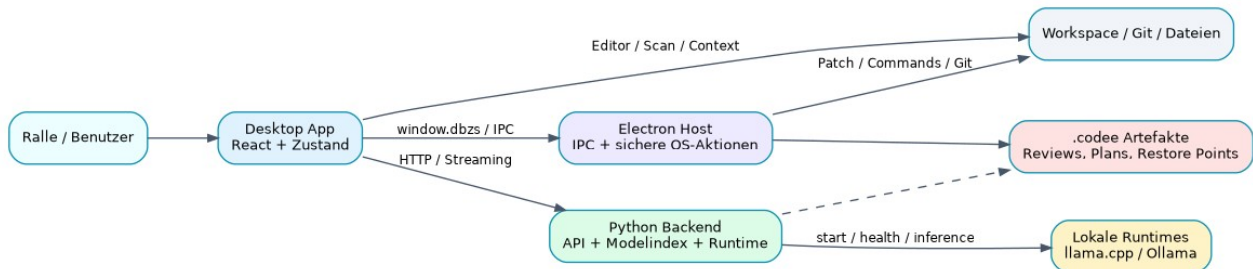


DBZS CODEE V4

Architektur- und Flussatlas des aktuellen Sourcecodes



Division By Zeros | Stand: 28. Juli 2026 | Repository HEAD: 4e1bb521878d614c493919f23e850ee85766e4fc

Zweck: Entwicklerorientierte Systemkarte für Codee, Codex, Cursor, MSC365 und zukünftige Maintainer.

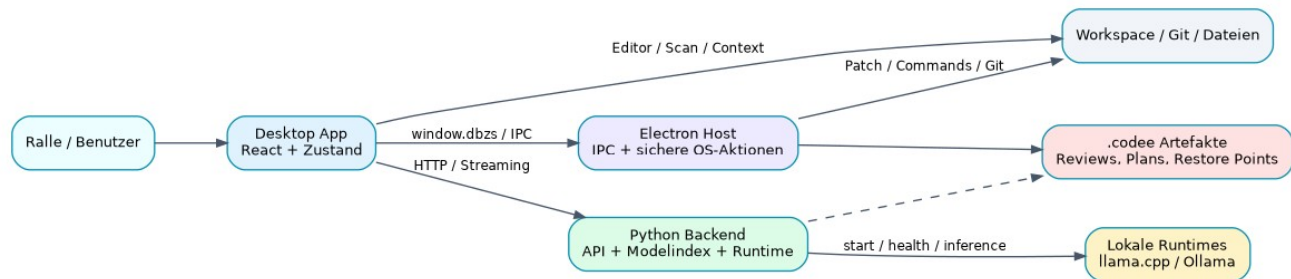
Inhaltsübersicht

1. 1 Gesamtarchitektur
2. 2 Prozess- und Vertrauensgrenzen
3. 3 Boot- und Startup-Flow
4. 4 Runtime Chat End-to-End
5. 5 Workflow- und Modellrouting
6. 6 Kontext, RAG und Tokenbudget
7. 7 Tools und Freigaben
8. 8 Chat-Actions und Follow-up-Lücke
9. 9 Patch, Diff, Apply, Tests und Rollback
10. 10 Repository Review und Remediation
11. 11 Modelindex und Discovery
12. 12 Runtime-Slots und Lazy Loading
13. 13 Multimodale Modelle und MMProj
14. 14 Zustand, Stores und Persistenz
15. 15 Fehler, Timeouts und Recovery
16. 16 Produktionsreife-Roadmap
17. 17 Contract- und Datenmodellübersicht
18. 18 Empfohlene Architekturregeln
19. 19 Prüfmatrix
20. 20 Fazit und Zielbild

Der Atlas ist bewusst hierarchisch aufgebaut: Systemebene -> Subsysteme -> Laufwege -> Zustände -> Produktionsreife.

1. Gesamtarchitektur

Codee besteht aus einer React-Desktopoberfläche, einem privilegierten Electron-Host, einem lokalen Python-Backend und dynamisch gestarteten lokalen Modellruntimes. Workspace-Dateien und .codee-Artefakte bilden die persistente Arbeitsgrundlage.



Status: REAL/PARTIAL - Kernprozesse laufen real; mehrere Golden-Path-Endstufen bleiben offen.

Kernpunkte

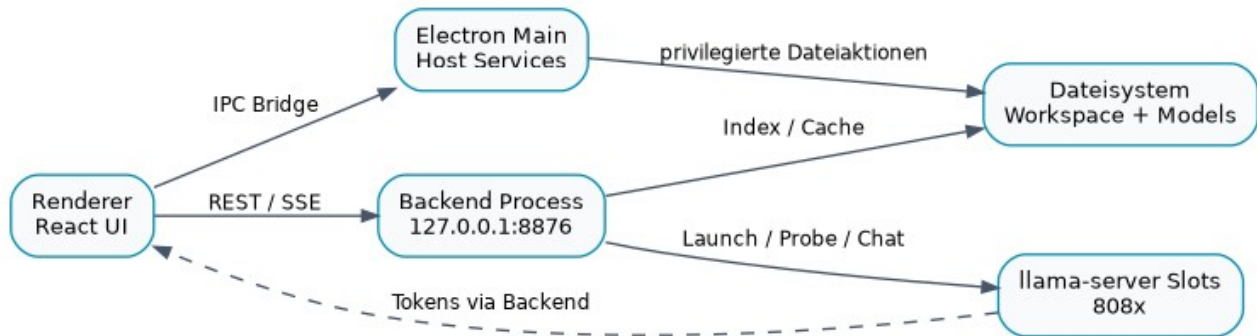
- Die React-Oberfläche bleibt unprivilegiert und kommuniziert über definierte Bridges.
- Electron besitzt die OS-nahen Fähigkeiten für Dateien, Git, sichere Befehle und Patch-Aktionen.
- Das Backend verwaltet Modelindex, Runtime-Provider, Health und lokale Inferenz.
- Lokale Modelle werden nicht pauschal beim Start geladen, sondern rollen- und aufgabenbezogen gestartet.

Wichtige Sourcecode-Pfade

apps/desktop/src/App.tsx
 apps/desktop/src/components/RuntimeChatTab.tsx
 apps/desktop/electron/
 backend/app/
 packages/shared/src/index.ts

2. Prozess- und Vertrauensgrenzen

Die Prozessgrenzen sind für Produktionsreife entscheidend: UI, Electron, Backend und Modellruntimes haben unterschiedliche Sicherheits- und Fehlerdomänen.



Kernpunkte

- Der Renderer darf keine beliebigen Systembefehle ausführen.
- Electron kapselt privilegierte Host-Aktionen und Sicherheitsprüfungen.
- Backend und Runtime-Prozesse können unabhängig ausfallen und müssen separat diagnostizierbar bleiben.
- Ports, PIDs, Endpoints und geladene Modell-IDs sind Teil des Runtime-Statusvertrags.

Wichtige Sourcecode-Pfade

`apps/desktop/electron/main.ts`

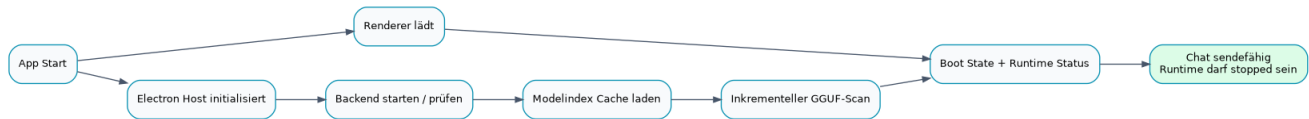
`apps/desktop/src/services/backendClient.ts`

`backend/app/api/`

`apps/desktop/src/services/runtimeSlotManager.ts`

3. Boot- und Startup-Flow

Der Boot-Pfad ist inzwischen auf Lazy Runtime ausgelegt: Der Chat darf sendefähig sein, sobald der Backendstatus bekannt ist; ein Arbeitsmodell muss nicht permanent resident sein.



Status: REAL - Boot, Rescan und echter lokaler Chat wurden UI-seitig verifiziert.

Kernpunkte

- Der Modelindex unterstützt Cache-Wiederherstellung und inkrementellen Scan.
- Ein einzelner defekter Katalog- oder Modelleintrag darf den Gesamtscan nicht abbrechen.
- Der Renderer leitet seine Statusanzeige aus einem zentralen Boot-State ab.
- Modelle und MMProj-Dateien sollen beim Start klassifiziert, aber noch nicht geladen werden.

Wichtige Sourcecode-Pfade

backend/app/models/index_service.py

backend/app/models/index_startup.py

apps/desktop/src/services/bootUiFormatter.ts

apps/desktop/src/components/RuntimeChatTab.tsx

4. Runtime Chat End-to-End

Eine Chatnachricht durchläuft Preflight, Workflow-Auflösung, Modellrouting, Runtime-Vorbereitung, Kontextaufbau, Streaming und Ergebnisverarbeitung.



Kernpunkte

- Der Store ist aktuell der zentrale Orchestrator und dadurch sehr groß.
- Preflight trennt fehlende Informationen, Continuation und echte Blocker.
- Streaming-Antworten werden mit Run- und Trace-Daten verknüpft.
- Artefakte wie Plan- oder Patchvorschläge werden nach der Modellantwort separat verarbeitet.

Wichtige Sourcecode-Pfade

apps/desktop/src/components/RuntimeChatTab.tsx

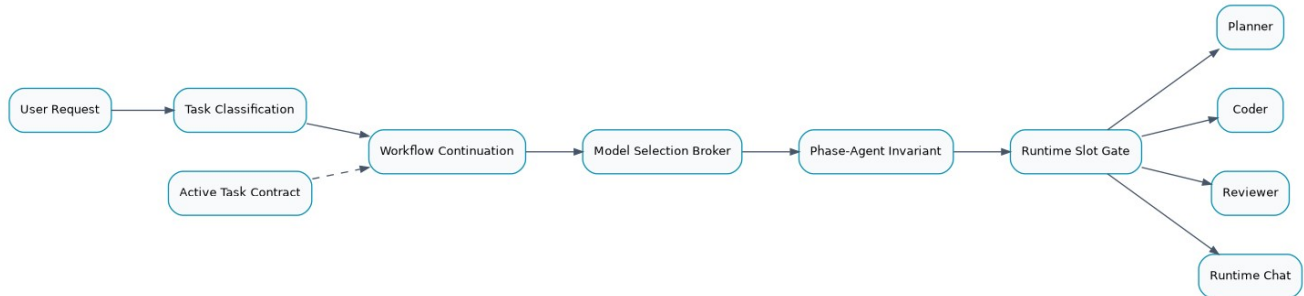
apps/desktop/src/stores/runtimeChatStore.ts

apps/desktop/src/stores/runtimeChatStoreSendPreflight.ts

apps/desktop/src/stores/runtimeChatStoreStreamingFinalization.ts

5. Workflow- und Modellrouting

Codee routet nicht nur nach Modellrolle, sondern nach Workflow, Phase, aktivem Task-Vertrag und Slot-Verfügbarkeit.



Status: REAL - Review-Routing funktioniert; Ergebnisqualität bleibt modellabhängig.

Kernpunkte

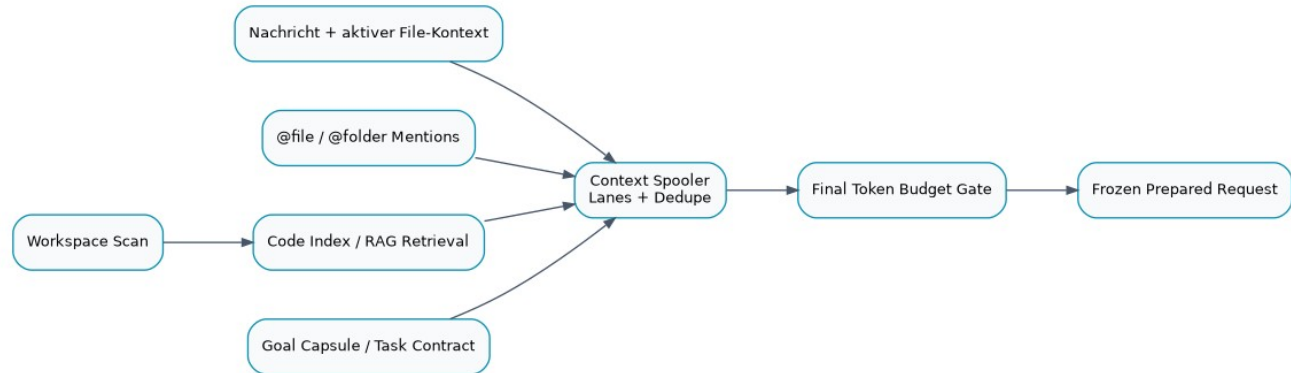
- Die zuletzt behobene Regression lag in der Fortsetzungslogik eines älteren offenen Task-Contracts.
- Phase-Agent-Invarianten verhindern fachlich falsche Zuweisungen.
- Implementierungsphasen werden auf den Coder normalisiert.
- Das Review-Routing wurde real durch einen UI-Lauf bestätigt.

Wichtige Sourcecode-Pfade

apps/desktop/src/runtime/workflow/workflowStateResolver.ts
 apps/desktop/src/services/modelSelectionBroker.ts
 apps/desktop/src/services/phaseAgentInvariant.ts
 apps/desktop/src/services/runtimeChat/workflowContinuationPhase.ts

6. Kontext, RAG und Tokenbudget

Der Kontextfluss kombiniert aktive Datei, Workspace-Scan, Mentions, RAG, Task-Vertrag und Goal Capsule. Der Context Spooler dedupliziert und priorisiert Quellen.



Kernpunkte

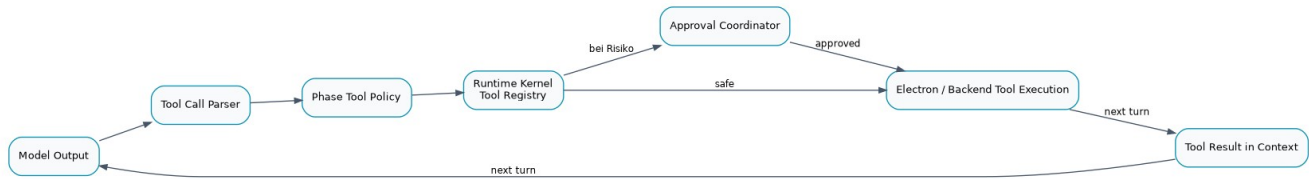
- Workspace-Dateien werden nicht ungefiltert vollständig in jeden Prompt gelegt.
- Tokenbudgets reservieren explizit Ausgaberaum für die jeweilige Aufgabe.
- Context Manifest und Retrieval Manifest machen Abdeckung und Drops sichtbar.
- Ein finaler Budget-Gate verhindert unkontrollierte Überläufe.

Wichtige Sourcecode-Pfade

apps/desktop/src/runtime/context/contextSpooler.ts
 apps/desktop/src/services/contextOrchestrator.ts
 apps/desktop/src/services/finalRequestTokenBudget.ts
 apps/desktop/src/services/runtimeChatContext.ts

7. Tools und Freigaben

Tool-Aufrufe werden aus Modellantworten extrahiert, gegen Phasenrichtlinien geprüft und über den Runtime Kernel ausgeführt. Risikobehaftete Aktionen benötigen Nutzerfreigaben.



Kernpunkte

- Toolkatalog und Toolbudget werden in den Promptvertrag eingebunden.
- Phase Tool Policies begrenzen erlaubte Werkzeuge je Workflowphase.
- Command- und Web-Aktionen nutzen die zentrale Approval-Infrastruktur.
- Toolergebnisse werden in einen Folgeturn zurückgeführt.

Wichtige Sourcecode-Pfade

apps/desktop/src/services/runtimeChatToolParser.ts
 apps/desktop/src/services/runtimeKernelService.ts
 apps/desktop/src/services/workflowPhaseToolPolicy.ts
 apps/desktop/src/services/approvalCoordinator.ts

8. Chat-Actions und Follow-up-Lücke

Die Chat-UI rendert vorhandene message.actions zuverlässig. Für normale erfolgreiche Antworten fehlt jedoch ein zentraler Post-Response Follow-up Action Builder.



Status: BLOCKED/FEHLEND - Generische Folgeaktionen sind noch nicht implementiert.

Kernpunkte

- Patch-, Command-, Web- und Review-Aktionen werden gezielt erzeugt.
- Normale Chat-, Analyse- oder Planungsergebnisse enden meist ohne Folgeangebot.
- Der statische Next-Preset ist in einem geschlossenen Detailbereich versteckt.
- Empfehlung: Required Actions und Suggested Follow-ups fachlich trennen.

Wichtige Sourcecode-Pfade

apps/desktop/src/components/runtime-chat/RuntimeChatMessageCard.tsx

apps/desktop/src/services/runtimeChatActionSelectors.ts

apps/desktop/src/services/runtimeAgentActions.ts

apps/desktop/src/stores/runtimeChatStorePatchActions.ts

9. Patch, Diff, Apply, Tests und Rollback

Dateiänderungen werden nicht direkt aus freiem Modelltext geschrieben. Sie durchlaufen Parsing, Preview, Freigabe, Restore Point, Apply und Validierung.



Status: PARTIAL - Services sind verdrahtet; vollständiger Golden Path ist noch nicht komplett UI-verifiziert.

Kernpunkte

- Der Patchvorschlag ist ein strukturierter Contract.
- Vor dem Apply wird ein Diff erzeugt und angezeigt.
- Ein Restore Point schützt vor fehlerhaften Änderungen.
- Nach Apply werden Tests, Rollback und Datei öffnen als Folgeaktionen angeboten.

Wichtige Sourcecode-Pfade

apps/desktop/src/services/agentOutputParser.ts
 apps/desktop/src/stores/runtimeChatStorePatchActions.ts
 apps/desktop/electron/patchPipelineService.ts
 apps/desktop/electron/restorePointService.ts

10. Repository Review und Remediation

Der Repository Review ist ein eigener orchestrierter Workflow mit Scope, Inventar, Batches, Hybridanalyse, Qualitätsbewertung und persistierten Artefakten.



Status: PARTIAL - Routing real, strukturierte Modellqualität derzeit Blocker.

Kernpunkte

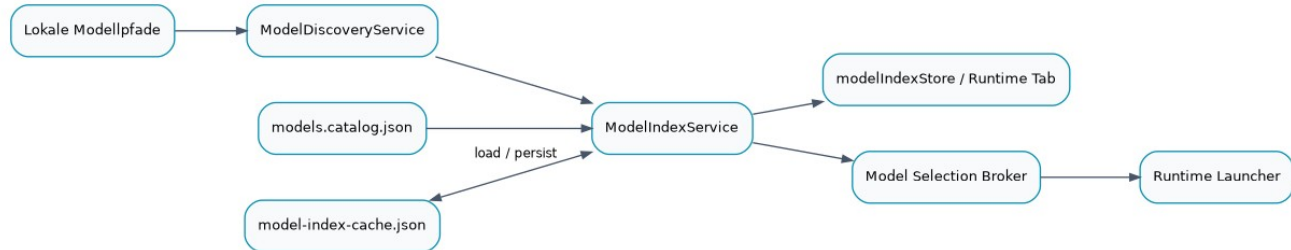
- Der strukturelle Reviewlauf wurde real ausgeführt und erzeugt .codee/reviews-Artefakte.
- Aktueller Blocker ist unzuverlässiger strukturierter Modellausgang: no_json_array.
- Heuristik kann degradierte Ergebnisse liefern, ersetzt aber keine belastbaren LLM-Findings.
- Findings können in einen Remediation-Workflow überführt werden.

Wichtige Sourcecode-Pfade

apps/desktop/src/services/repositoryReview/
 apps/desktop/src/services/reviewRemediation.ts
 apps/desktop/src/services/reviewRemediationSelection.ts
 docs/audits/GOLDEN_PATH_VERIFICATION_2026-07-28-ui.md

11. Modelindex und Discovery

Der Modelindex scannt lokale GGUF-Dateien, berücksichtigt Katalog- und Runtimeinformationen, validiert Header, cached Signaturen und leitet Rollen sowie Kompatibilität ab.



Status: REAL/PARTIAL - Scan ist real; Capability-Zertifizierung und Artefakttrennung fehlen.

Kernpunkte

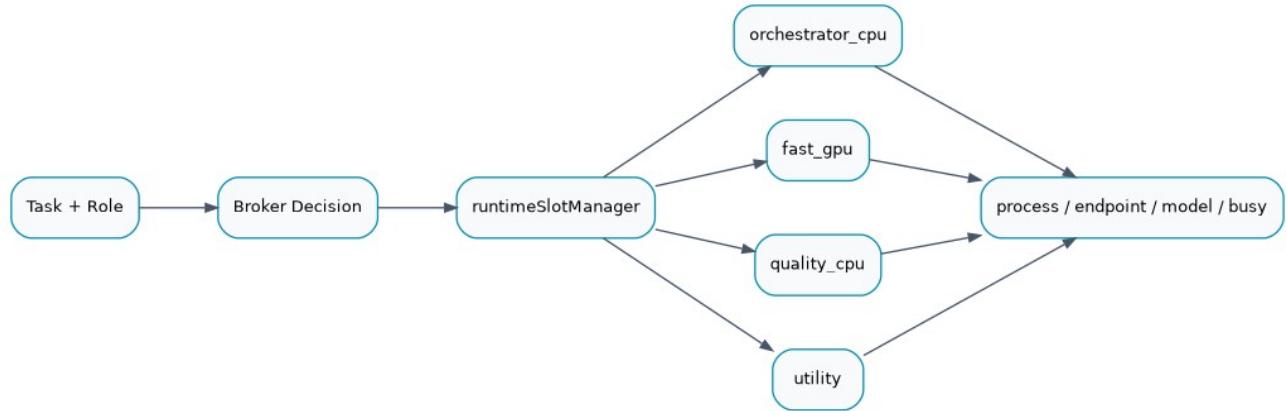
- Der reale Rescan erfasste 364 Modelle ohne Regression.
- Dateinamenheuristiken sind aktuell noch zu wichtig für Capability-Ableitungen.
- Support-Artefakte wie MMProj müssen stärker von startbaren Modellen getrennt werden.
- Structured Output Reliability muss als eigene Capability aufgenommen werden.

Wichtige Sourcecode-Pfade

backend/app/models/index_service.py
 backend/app/models/discovery.py
 backend/app/models/schemas.py
 apps/desktop/src/stores/modelIndexStore.ts

12. Runtime-Slots und Lazy Loading

Der Runtime Slot Manager trennt Orchestrator, schnelle GPU-Arbeit, CPU-Qualität und Utility-Aufgaben. Slots werden anhand von Rolle, Ressourcenrisiko und Bereitschaft ausgewählt.



Status: PARTIAL - Lazy Runtime ist aktiv; Crash- und Modellwechselstabilität offen.

Kernpunkte

- Der Orchestrator-Slot kann ein sehr kleines Routingmodell resident halten.
- Arbeitsmodelle werden bei Bedarf geladen und nach Inaktivität entladen.
- Slot Readiness prüft Prozess, Endpoint, Modell und Busy-Zustand.
- Ein unerklärter App-/Backend-Absturz nach Modellwechsel muss noch root-caused werden.

Wichtige Sourcecode-Pfade

apps/desktop/src/services/runtimeSlotManager.ts
 apps/desktop/src/services/lazyRuntimePolicy.ts
 packages/shared/src/index.ts
 backend/app/runtime/

13. Multimodale Modelle und MMProj

MMProj-Dateien sind nicht eigenständig startbar. Codee muss sie als Utility-Support-Artefakte erfassen, mit kompatiblen Basismodellen paaren und erst nach einer echten Media-Probe freigeben.



Status: GEPLANT - Erkennung teilweise vorhanden, Pairing und Runtime-Probe fehlen.

Kernpunkte

- MMProj allein wird als Orphan Support Artifact sichtbar, aber nicht geroutet.
- Same-Folder-Erkennung ist ein starkes Kandidatensignal, jedoch kein Beweis.
- Verifizierter Start benötigt --model und --mmproj in derselben Runtime.
- Für Coding mit Screenshot ist zunächst Vision Context Pack -> Coder die sicherste Pipeline.

Wichtige Sourcecode-Pfade

backend/app/models/index_service.py

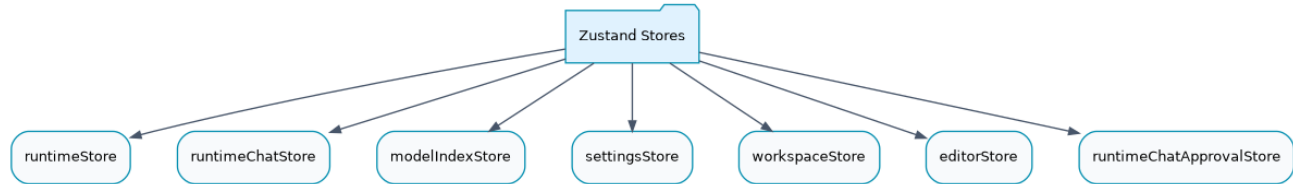
apps/desktop/src/services/modelSelectionBroker.ts

apps/desktop/src/utls/modelUtils.ts

geplant: runtimeChatMultimodalPairing.ts

14. Zustand, Stores und Persistenz

Die Desktop-App verwendet mehrere spezialisierte Zustand-Stores. Der Runtime Chat Store bündelt derzeit sehr viele Verantwortlichkeiten und ist der größte Architektur-Hotspot.



Kernpunkte

- runtimeStore verwaltet den laufenden Modellprozess.
- modelIndexStore verwaltet den Index, nicht die Runtime.
- runtimeChatApprovalStore und Agent Actions bilden Freigaben ab.
- Workspace-, Editor- und Git-Store müssen nach Patches synchronisiert werden.

Wichtige Sourcecode-Pfade

`apps/desktop/src/stores/runtimeChatStore.ts`

`apps/desktop/src/stores/runtimeStore.ts`

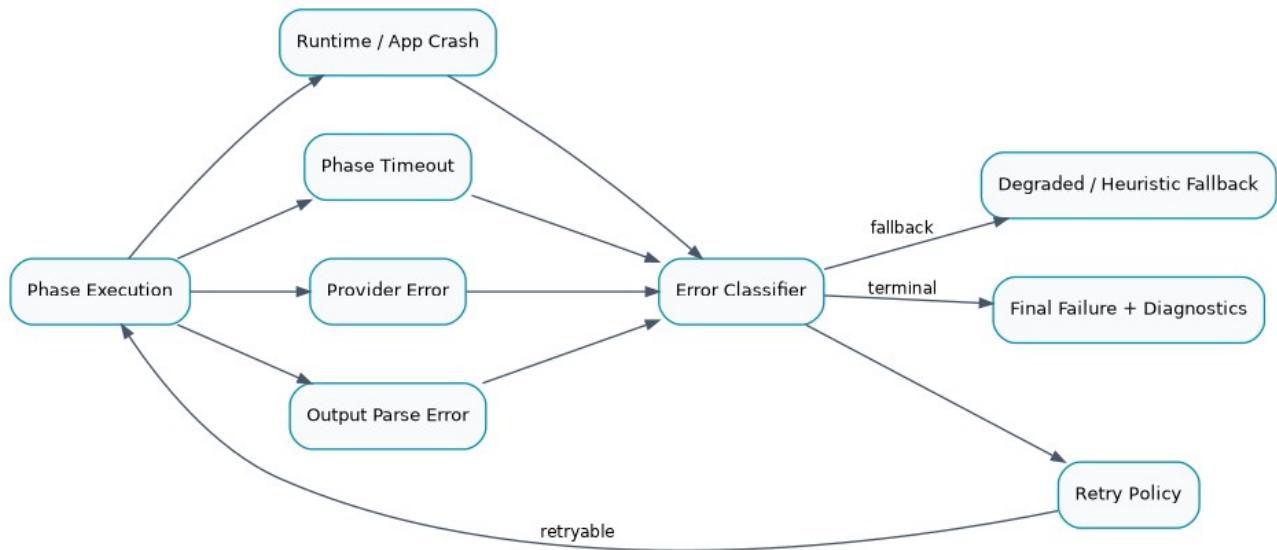
`apps/desktop/src/stores/modelIndexStore.ts`

`apps/desktop/src/stores/workspaceStore.ts`

`apps/desktop/src/stores/editorStore.ts`

15. Fehler, Timeouts und Recovery

Fehler können in jeder Phase auftreten: Provider, Timeout, Parser, Runtime, Tool oder App. Codee klassifiziert und entscheidet zwischen Retry, Fallback, Degraded Mode und terminalem Abbruch.



Kernpunkte

- Phasenbezogene Timeouts verhindern endlose Runs.
- Providerfehler und leere Antworten können kontrolliert wiederholt werden.
- Parserfehler wie `no_json_array` benötigen Capability-Diagnostik, nicht nur Retry.
- Crashlogs und Run-Diagnosen müssen korreliert und exportierbar sein.

Wichtige Sourcecode-Pfade

apps/desktop/src/services/runtimeChatErrorClassifier.ts
 apps/desktop/src/services/runtimePhaseTimeouts.ts
 apps/desktop/src/services/runtimeRunFinalization.ts
 apps/desktop/src/stores/runtimeChatStoreSendFailure.ts

16. Produktionsreife-Roadmap

Die nächste Entwicklung sollte Stabilität und messbare Modellfähigkeiten vor großen UI-Erweiterungen priorisieren.



Kernpunkte

- P0: Structured Output Reliability und Modellwechsel-Crash.
- P1: Support-Artefakte und MMProj-Pairing im Modelindex.
- P2: Runtime Tab als Model Control Center auf stabile Contracts bauen.
- P3: Runtime-Probes, Benchmarks und Capability-Zertifizierung.
- P4: Multimodales Routing und Vision Context Packs.
- P5: Golden Path, Wiederholungsläufe und Installer für PERSONAL_STABLE.

Wichtige Sourcecode-Pfade

TODO.md

HANDOVER.md

docs/audits/GOLDEN_PATH_VERIFICATION_2026-07-28-ui.md

17. Contract- und Datenmodellübersicht

Die wichtigsten Architekturgrenzen werden durch Shared Contracts stabilisiert. Der Atlas empfiehlt, neue Funktionen additiv einzuführen und bestehende Aufrufer kompatibel zu halten.

Contract	Verantwortung
RuntimeStatus	Provider, Modell-ID, PID, Port, Endpoint, Zustand
IndexedModel	Pfad, Artefakttyp, Capability-Hinweise, Rolle, Kompatibilität
RuntimeChatMessage	Rolle, Inhalt, Actions, Proposal- und Trace-Referenzen
RuntimeChatRun	Workflow, Phase, Agent, Modell, Events, Outcome, Diagnosen
AgentPlanProposal	strukturierter Plan mit Schritten und Status
AgentPatchProposal	strukturierte Dateiänderungen und Validierungsbefehle
ChatActionRequest	Freigabe- und Transportaktion mit Workspace-Bindung
RepositoryReviewFinding	Severity, Kategorie, Pfad, Evidenz, Empfehlung

Additiv geplante Contracts

- ModelSupportArtifact - MMProj, LoRA und Adapter als nicht eigenständig startbare Artefakte.
- MultimodalPair - verifizierte Verbindung zwischen Basismodell und Projector.
- ModelCapabilityProfile - Evidenz, Confidence, Testhistorie und Routingfreigabe.
- SuggestedFollowUpAction - dialogische Fortsetzungen getrennt von Sicherheitsfreigaben.

18. Empfohlene Architekturregeln

21. Bestehende öffentliche Store- und IPC-Verträge nur additiv erweitern.
22. Support-Artefakte niemals aufgrund eines Kompatibilitätsstrings als Modell behandeln.
23. Modellnamen und Dateinamen sind Hinweise, keine Capability-Beweise.
24. Routing nutzt nur verifizierte Mindestfähigkeiten der jeweiligen Rolle.
25. Multimodalität und Fachrolle getrennt modellieren: IMAGE + CODER statt VISION als Universalrolle.
26. Normale Chat-Folgeaktionen von Approval-Actions trennen.
27. Kein großer Model- oder Vision-Load beim App-Start.
28. Jeder privilegierte Vorgang benötigt Workspace-Bindung, Version und Stale-Schutz.
29. Run-Ergebnisse müssen Diagnose und ehrlichen Degraded-Status enthalten.
30. Produktionsreife wird über wiederholbare Golden-Path-Läufe bewiesen, nicht nur über Unit Tests.

Leitformel: Index findet - Capability Engine beweist - Broker routet - Runtime lädt - Workflow führt aus - Approval schützt - Tests verifizieren.

19. Prüfmatrix des aktuellen Entwicklungsstands

Bereich	Status	Begründung
Boot / Backend	REAL	Echter UI- und Servicebetrieb verifiziert
Model Rescan	REAL	364 Modelle ohne Regression gescannt
Lokaler Chat	REAL	Gemma-3-1B lokal im UI genutzt
Workflow Routing	REAL	Reviewer-Zuweisung nach Fix verifiziert
Repository Review Struktur	REAL/PARTIAL	Artefakte erzeugt, LLM-Findings unbrauchbar
Structured Output	BLOCKED	no_json_array / generation_failed
Patch Preview / Apply Services	PARTIAL	verdrahtet, End-to-End noch nicht vollständig
Tests / Rollback Golden Path	OPEN	UI-Verifikation noch ausstehend
Model Control Center	GEPLANT	Runtime Tab noch einfache Tabelle
MMProj Pairing	GEPLANT	keine verifizierte Pairing Engine
Generische Chat Follow-ups	FEHLEND	keine Post-Response Action Generation
PERSONAL_STABLE	NOCH NICHT	Wiederholungsläufe und Installer offen

20. Fazit und Zielbild

Codee besitzt inzwischen eine reale, ungewöhnlich breite technische Basis: lokaler Chat, dynamische Runtimes, Workflow-Routing, Context Management, Tool- und Approval-Infrastruktur, Repository Review sowie eine sichere Patch-Pipeline. Die Architektur ist jedoch an mehreren Stellen schneller gewachsen als ihre einheitlichen Contracts und Benutzerflüsse.

Die drei wichtigsten nächsten Architekturentscheidungen

31. Structured Output Reliability als messbare Modellfähigkeit etablieren und den aktuellen no_json_array-Blocker beseitigen.
32. Modelindex um klare Support-Artefakte, MMProj-Pairing und Capability-Evidenz ergänzen.
33. Chat nach jeder Antwort mit deterministischen, kontextbezogenen Folgeaktionen ausstatten.

Zielbild

Codee erkennt die Aufgabe, lädt das richtige lokale Modell, beschafft nur den nötigen Kontext, führt sichere Tools aus, schlägt konkrete nächste Schritte vor und kann jede Änderung testen sowie zurückrollen.

Dieser Atlas ist eine schematische Entwicklerkarte. Er ersetzt keine API-Referenz, schafft aber die gemeinsame Architekturgrundlage, auf der Codex, Cursor, MSC365 und Codee selbst konsistent weiterarbeiten können.

Repository-Referenz: [devdbzemusic/dbzs-codee-v4 @ 4e1bb521878d614c493919f23e850ee85766e4fc](#)